

Topics in Software Engineering – **Software Reengineering**

By

Dr. Junaid Akram

Assistant Professor, Department of Computer Science COMSATS (Lahore)

PhD (Software Engineering) Tsinghua University

Post Doctrate (Software Testing) University of Luxembourg

Introduction to Reengineering

Basic Concepts and Preliminaries

Overview of the course

- Software Change
- Software Evolution
- Software Maintenance
- Software Evolution Models and Processes
- Reengineering
- Legacy Systems
- Impact Analysis
- Refactoring
- Program Comprehension
- Software Reuse

Software Change

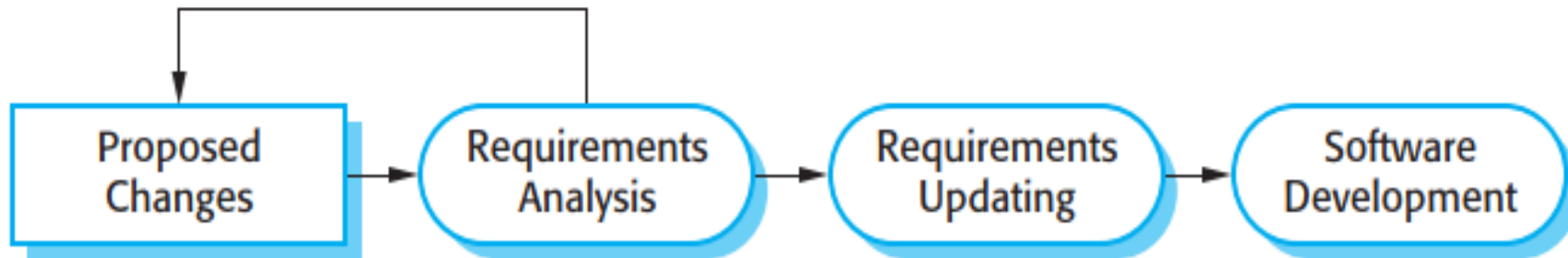
- Software **development does not stop when a system is delivered** but continues throughout the lifetime of the system.
- After a system has been deployed, it would **require change** if it is to remain useful.
- Business changes generate **new requirements** for the existing software.
- Parts of the software may have to be modified to **correct errors** that are found in operation,
- Adapt it for changes to its hardware and software platform, and to improve its

Software Change

- Software change is inevitable:
 - New requirements emerge when the software is used
 - Errors must be repaired
 - New computers and equipment are added to the system
 - The performance or reliability of the system may have to be improved
- A key problem for all organizations is implementing and managing change to their existing software systems

Change Implementation

During the **evolution process**, the requirements are analyzed in detail and implications of the **changes emerge** that were not apparent in the earlier change analysis process. This means that the proposed changes may be modified and further customer discussions may be required before they are implemented.



Types of Change

Based on the type of changes required, system characteristics are divided into groups:

re-code , re-design , re-specify and re-think

Recode:

- Characteristics of the source program are changed by re-coding it.
- A program is transformed into a **different language**.
- Examples of rephrasing scenarios are **normalization, optimization, refactoring, and renovation**.

Redesign:

- The design characteristics of the software are altered by re-designing the system. Common changes to the software design include:
 - Restructuring the **architecture**;
 - Modifying the **data model** of the system; and
 - Replacing a procedure or an **algorithm** with a more efficient one.

Cont..

Respecify:

- This means **changing the requirement** characteristics of the system in two ways:
 - change the **form** of the requirements
 - change the **scope** of the requirements

Rethink:

- Re-thinking a system means **manipulating the concepts** of an existing system that operates in a different problem domain.
- It involves **changing the conceptual characteristics** of the system
- Example: Moving from the development of an ordinary cellular phone to the development of

Evolution Versus Maintenance

- The terms evolution and maintenance are used **interchangeably**. However, there is a semantic difference.
- **Lowell Jay** Arthur distinguish the two terms as follows:
 - “**Software maintenance means to preserve from failure or decline.**”
 - “**Software evolution means a continuous change from lesser, simpler, or worse state to a higher or better state.**”
- Maintenance is considered to be set of planned activities
- Evolution concern whatever happens to a system over time

Software Evolution: Laws of Lehman



Continuing change (1st) – A system will become progressively less satisfying to its user over time, unless it is continually adapted to meet new needs.



Increasing complexity (2nd) – A system will become progressively more complex, unless work is done to explicitly reduce the complexity.



Self-regulation (3rd) – The process of software evolution is self regulating with respect to the distributions of the products and process artifacts that are produced.



Conservation of organizational stability (4th) – The average effective global activity rate on an evolving system does not change over time, that is the average amount of work that goes into each release is about the same.

Software Evolution: Laws of Lehman



Conservation of familiarity (5th) – The amount of new content in each successive release of a system tends to stay constant or decrease over time.



Continuing growth (6th) – The amount of functionality in a system will increase over time, in order to please its users.

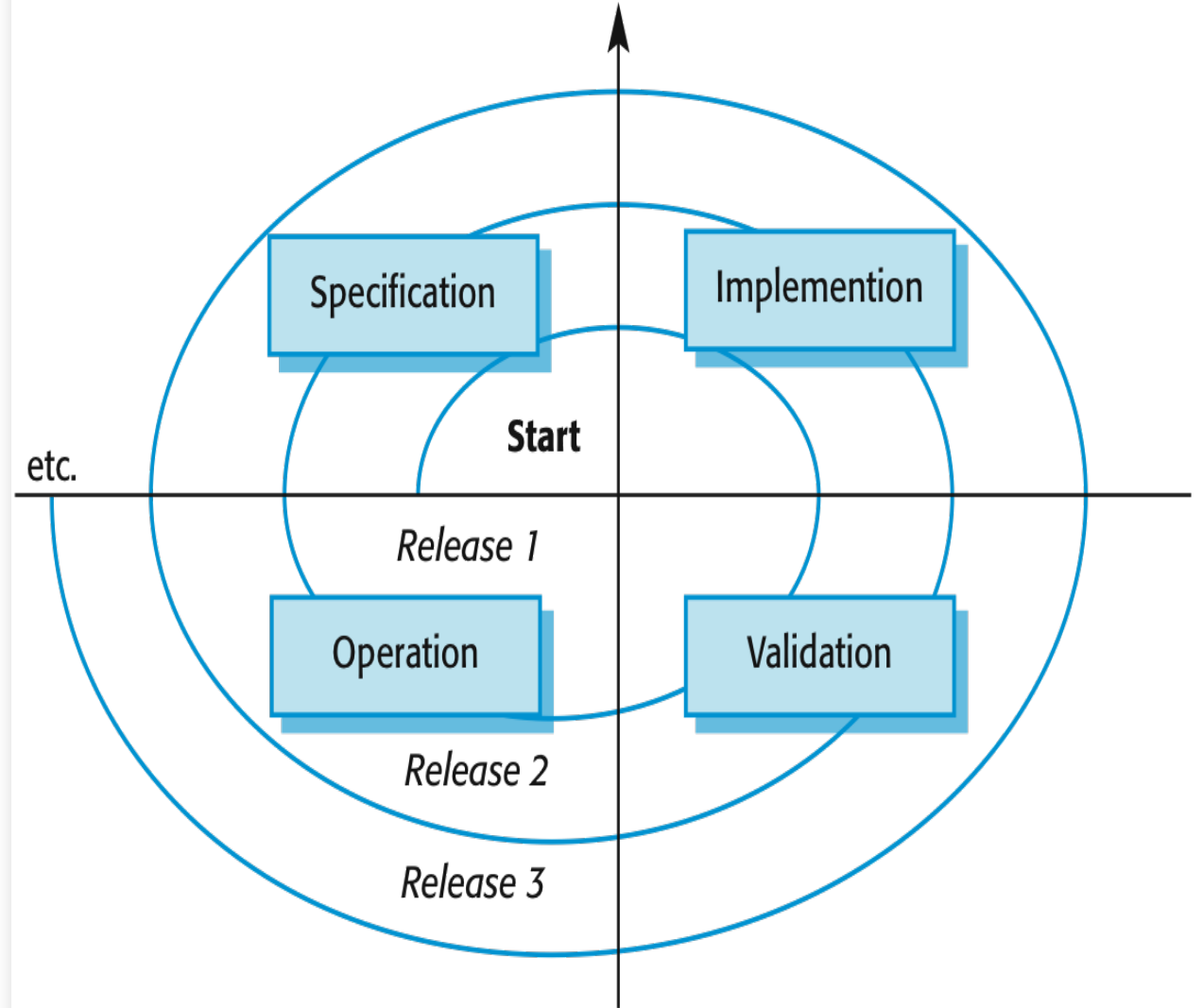


Declining quality (7th) – A system will be perceived as losing quality over time, unless its design is carefully maintained and adapted to new operational constraints.



Feedback system (8th) – Successfully evolving a system requires recognition that the development process is a multi-loop, multi-agent, multi-level feedback system.

Spiral Model of Development and Evolution



Software Maintenance

- There will always be defects in the delivered software application because software defect removal and quality control are not perfect.
- Therefore, software maintenance is needed to repair these defects
- E. Burton Swanson defined three type of software maintenance:
 - Corrective,
 - Adaptive
 - Perfective

Cont..

- Swanson definition was later updated in the standard for software engineering – ISO/IEC 14764.
- Introduced a fourth category called preventive maintenance.
- Some researchers and developers view preventive maintenance as a subset of perfective maintenance.
- IEEE/EIA 1219 organizes the maintenance process in seven phases:
 - problem identification, analysis, design, implementation, system test, acceptance test and delivery

Software Maintenance Models and Processes

Software maintenance have its own software maintenance life cycle (SMLC) model.

Three popular SMLC models:

- Staged model of maintenance and evolution.
- Iterative models.
- Change mini-cycle models.

Reengineering

- Reengineering is done to transform an existing “**lesser or simpler**” system into a new “**better**” system.
- Reengineering is the **examination, analysis** and **restructuring** of an existing software system to reconstitute it in a new form.
- Chikofsky and Cross II defines reengineering as:
“the examination and alteration of a subject system to reconstitute it in a new form and the subsequent implementation of the new form.”

When to re-engineer

- When **system changes** are mostly confined to part of the system then re-engineer that part
- When **hardware or software support becomes obsolete**
- When **tools to support** re-structuring are available

Re-engineering advantages

- **Reduced risk:** There may be development problems, staffing problems and specification problems
- **Reduced cost:** The cost of re-engineering is often significantly less than the costs of developing new software

Re-engineering cost factors

Cost factors depends on

- The *quality* of the software to be re-engineered
- The *tool support* available for re-engineering
- The *extent of the data conversion* which is required
- The availability of *expert staff* for re-engineering

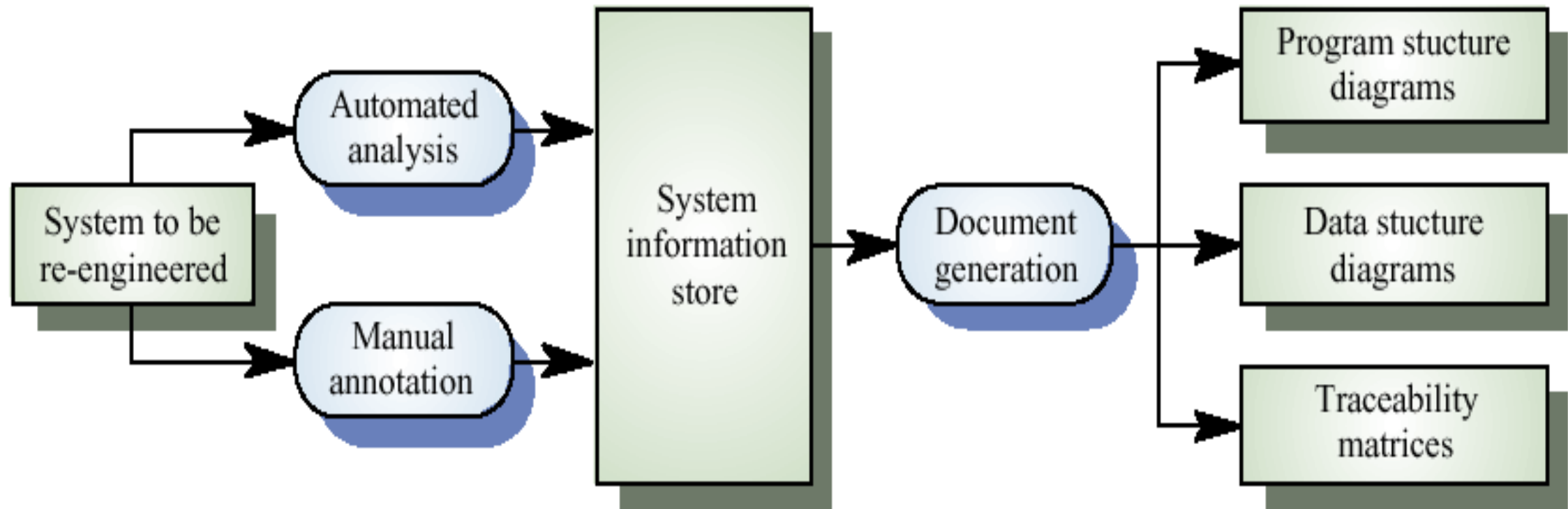
Reengineering

Jacobson and Lindstorm defined following formula:

Reengineering = Reverse engineering + Δ + Forward engineering.

- **Reverse engineering** is the activity of defining a more abstract, and easier to understand, representation of the system. The core of reverse engineering is the process of examination, not a process of change
- **Forward engineering** is the traditional process of moving from high-level abstraction designs to the physical implementation of the system.
- Δ captures alteration that is **change** of the system.

The Reverse Engineering Process



Goals of Reengineering

- **Unbundling**

Split a monolithic system into parts that can be separately marketed

- **Performance**

“First do it, then do it right, then do it fast”

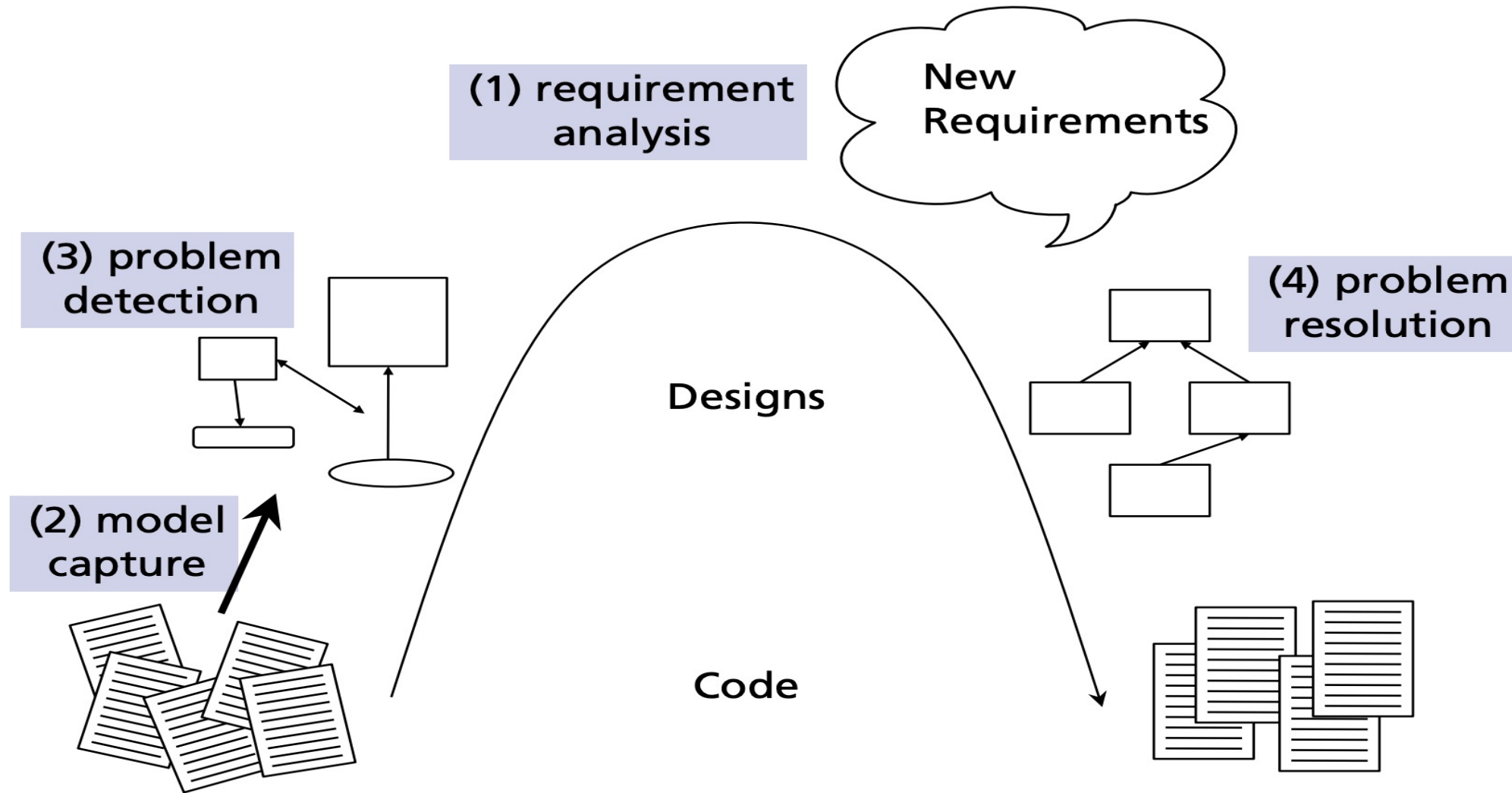
- **Design extraction**

To improve maintainability, portability, etc.

- **Exploitation of New Technology**

I.e., new language features, standards, libraries, etc.

Reengineering Life-Cycle



A General Model for Software Reengineering

- The model proposed by Eric J. Byrne suggests that reengineering is a sequence of three activities:
 - reverse engineering, re-design, and forward engineering

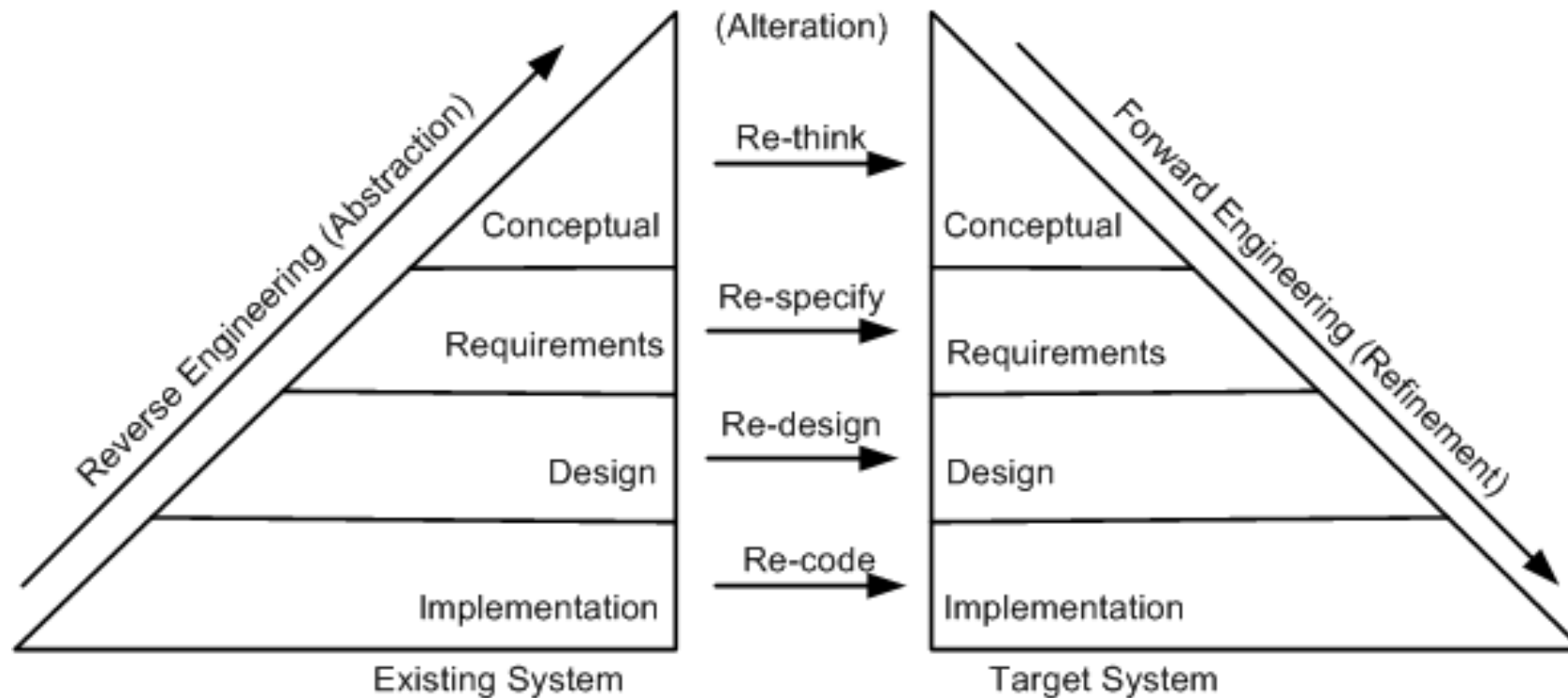


Figure: General model of software reengineering © IEEE, 1992

Source code translation

- Involves converting the code from one language (or language version) to another e.g. FORTRAN to C
- May be necessary because of:
 - Hardware platform update
 - Staff skill shortages
 - Organisational policy changes
- Only realistic if an automatic translator is available

Data re-engineering

- Involves **analysing** and **reorganising** the data structures (and sometimes the data values) in a program
- May be part of the process of migrating from a file-based system to a DBMS-based system or changing from one DBMS to another
- Objective is to create a **managed data environment**

Legacy System

- A **legacy system is an old system** which is valuable for the company which often developed and owns it.
- It is the phase out stage of the software evolution model of Rajlich and Bennet described earlier.
- Bennett used the following definition:
 - “large software systems that we don’t know how to cope with but that are vital to our organization.”
- Similarly, Brodie and Stonebraker: define a legacy system as
 - “any information system that significantly resists modification and evolution to meet new and constantly changing business requirements.”

Legacy System

There are a number of options available to manage legacy systems.

- **Freeze:** The organization decides no further work on the legacy system should be performed.
- **Outsource:** An organization may decide that supporting software is not core business, and may outsource it to a specialist organization offering this service.
- **Carry on maintenance:** Despite all the problems of support, the organization decides to carry on maintenance for another period.
- **Discard and redevelop:** Throw all the software away and redevelop the application once again from scratch.
- **Wrap:** It is a black-box modernization technique – surrounds the legacy system with a software layer that hides the unwanted complexity of the existing data, individual programs, application systems, and interfaces with the new interfaces.
- **Migrate:** Legacy system migration basically moves an existing, operational system to a new platform, retaining the legacy system's functionality and causing minimal disruption to the existing operational business environment as possible.

Impact Analysis

Two additional notions related to impact analysis are very common among practitioners:

- **Ripple effect.**
- **Change propagation.**
- Ripple effect analysis measures the impact, or how likely it is that a change to a particular module may cause problems in the rest of the program.
- Measuring ripple effect can provide knowledge about the system as a whole through its evolution:
 - how much its complexity has increased or decreased since the previous version,
 - how complex individual parts of a system are in relation to other parts of the system, and
 - to look at the effect that a new module has on the complexity of a system as a whole when it is added.
- The change propagation activity ensures that a change made in one component is propagated properly throughout the entire system.

Refactoring

- Refactoring is the process of making a change to the internal structure of software to make it easier to understand and cheaper to modify without changing its observable behavior.
- It is the object-oriented equivalent of restructuring.
- Refactoring, which aims to improve the internal structure of the code, achieve through the removal of duplicate code, simplification, making code easier to understand, help to find defects and adding flexibility to program faster.
- There are two aspects of the above definition:
 - It must preserve the “observable behavior” of the software system (through regression).
 - To improve the internal structure of a software system (improve maintainability).

Program Comprehension

- Program understanding or comprehension is
 - “the task of building mental models of an underlying software system at various abstraction levels, ranging from models of the code itself to ones of the underlying application domain, for software maintenance, evolution and re-engineering purposes.”
- A mental model describes a programmer’s mental representation of the program to be comprehended.
- Program comprehension deals with the cognitive processes involved in constructing a mental model of the program.
- A common element of such cognitive models is generating hypotheses and investigating whether they hold or must be rejected.
 - Hypotheses are a way to understand code in an incremental manner.
 - After some understanding of the code, the programmer forms a hypothesis and verifies it by reading code.
 - By continuously formulating new hypotheses and verifying them, the programmer understands more and more code and in increasing details.

Program Comprehension

- Several strategies can be used to arrive at relevant hypotheses such as:
 - **bottom up (starting from the code).**
 - **top down (starting from high-level goal).**
 - opportunistic combinations of the two.
- A strategy is formulated by identifying actions to achieve a goal.
- Strategies guide two mechanisms, namely chunking and cross-referencing to produce higher-level abstraction structures.
 - Chunking creates new, higher level abstraction structures from lower level structures.
 - Cross-referencing means being able to make links elements of different abstraction levels.
- Chunking and cross-referencing helps in building mental model of the program under study at different levels of abstractions.

Software Reuse

- Software reuse was introduced by Dough McIlroy in his 1968 seminal paper:
 - The development of an industry of reusable source-code software components and the industrialization of the production of application software from off-the-shelf components.
- Other significant early reuse research developments include:
 - Program families (David Parnas).
 - Domain analysis (Jim Neighbors).
- **Program families** are sets of programs whose common properties are so extensive that it becomes advantageous to study the common properties of these programs before analyzing individual differences.
- Whereas **domain analysis** is an activity of identifying objects and operations of a class of similar systems in a particular problem domain.

Cont..

- Software reuse is using existing artifacts or software knowledge during the construction of a new software system.
 - Reusable assets can be either reusable artifacts or software knowledge.
- Capers Jones identified four types of reusable artifacts:
 - **data reuse**, involving a standardization of data formats,
 - **architectural reuse**, which consists of standardizing a set of design and programming conventions dealing with the logical organization of software,
 - **design reuse**, for some common business applications, and
 - **program reuse**, which deals with reusing executable code.
- Software reuse of previously written code is a way to increase
 - software development productivity.
 - quality of the software.
- The cost savings during maintenance as a consequence of reuse are nearly twice the corresponding savings during development.

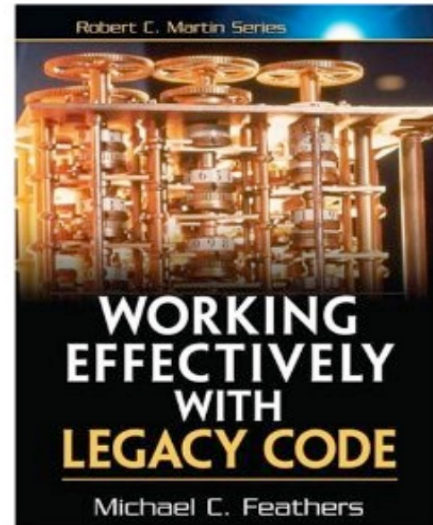
Cont..

- For software component to be reusable, it must exhibit the following characteristics that directly encourage its use in similar situation:
 - **Environmental independence** - The components can be reused irrespective of the environment from which they were originally captured.
 - **High cohesion** - The components that implement a single operation or set of related operations.
 - **Loose coupling** - The components that have minimal links to other components.
 - **Adaptability** - The components that are adaptable so they can be customized to fit a range of similar situations.
 - **Understandability** - The components which are easily understandable that users can quickly interpret functionality.
 - **Reliability** - The components that are error-free.
 - **Portability** - The components that are not restricted in terms of the software or hardware environment they operate in.

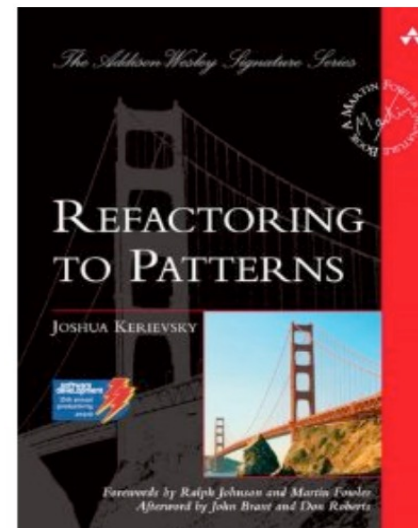
Reading material



Object-Oriented Reengineering Patterns
Serge Demeyer, Stéphane Ducasse, and Oscar Nierstrasz
free copy from: <http://scg.unibe.ch/download/oorp/>

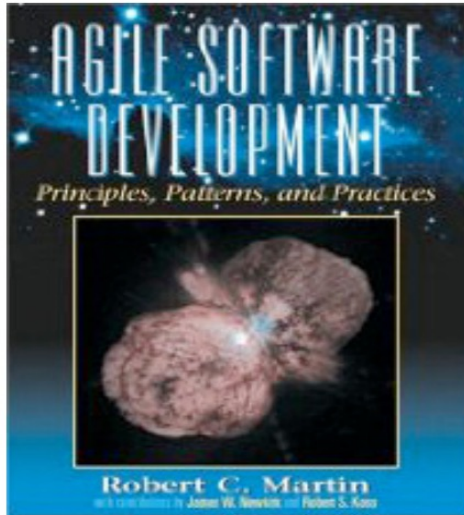


Working Effectively with Legacy Code
Michael Feathers, Prentice Hall, 1 edition, 2004

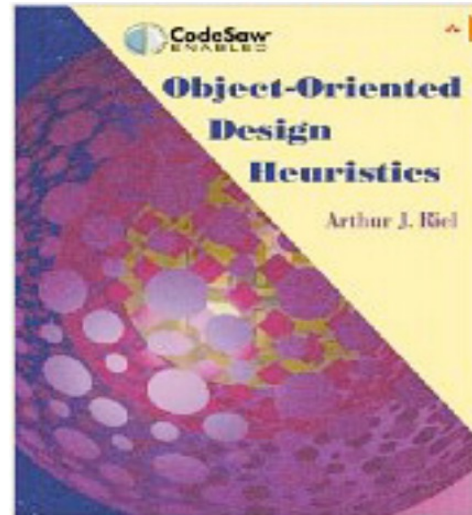


Refactoring to Patterns
Joshua Kerievsky, Addison-Wesley Professional, 2004
I will provide copies of selected chapters

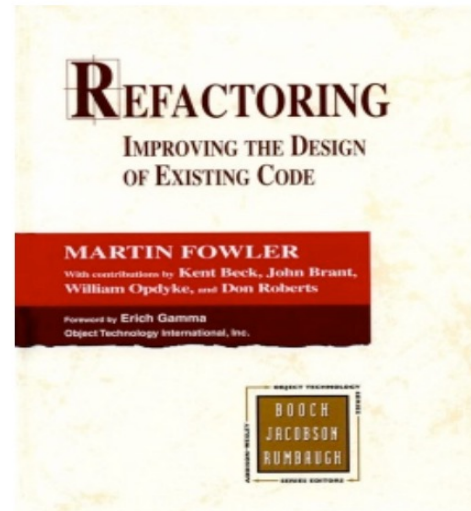
Additional reading



Agile Software Development: Principles Patterns, and Practices
Robert C. Martin, Prentice Hall



Object-Oriented Design Heuristics
Arthur J. Riel, Prentice Hall, 1 edition, 1996



Refactoring: Improving the Design of Existing Code
Martin Fowler, Addison-Wesley Professional, 1999

Thanks for your attention!

Any Question?



Email me on : junaidakram@cuilahore.edu.pk